Weekly Report

L-BFGS (Limited memory BFGS)

Hansu Kim
kimhansu1206@kentech.ac.kr

October 22, 2026

Contents

1 Background **1**
1.1 Newton's Method 1
1.2 BFGS optimization 2

2 L-BFGS **3**
2.1 Background 3
2.2 Method 3
2.3 L-BFGS Derivation and Two-Loop Recursion 3
2.3.1 Inverse Hessian Update Formulation ($m = 1$) 3
2.3.2 Base Scaling Factor γ_{k+1} 3
2.3.3 Step-by-Step Algebraic Decomposition 4
2.3.4 Full Iteration Step Definition 4

1 Background

In optimization, first-derivative information indicates the **direction to update**. The second derivative information implies **step size to move**.

1.1 Newton's Method

Newton's method uses second derivative information to find optimal points. To illustrate a simple case, Let's consider a 1-dimensional function $f(x)$ and its Taylor expansion at point $x_k$in Equation 1 [1].

$$f(x_k + \Delta x) \approx f(x_k) + f'(x_k)\Delta x + \frac{1}{2}f''(x_k)(\Delta x)^2,\ x_k \in \mathbb{R} \tag{1}$$

The objective of the second derivative is to find the optimal Δx to minimize the difference between the original function and the approximation in Equation 2.

$$\begin{aligned} &\frac{d}{d(\Delta x_k)}\left[f(x_k) + f'(x_k)\Delta x + \frac{1}{2}f''(x_k)(\Delta x)^2\right] = f'(x_k) + f''(x_k)\Delta x_k = 0 \\ &\Delta x_k = -\frac{f'(x_k)}{f''(x_k)} \end{aligned} \tag{2}$$

Therefore, in Newton's method, the next steps are defined by the prior step and its step size in Equation 3

$$x_{k+1} = x_k + \Delta x = x_k - \frac{f'(x_k)}{f''(x_k)} \tag{3}$$

For extension to a multi-variable function, Let's consider $x \in \mathbb{R}^n$ (since the target function may be premised as a scalar function; for the vector field case, a linear transform, like a linear combination of each part, may be applied to construct a scalar target function to be optimized). Then, for each step, the gradient can be defined in Equation 4.

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1} & \frac{\partial f}{\partial x_2} & \cdots & \frac{\partial f}{\partial x_n} \end{bmatrix}^T \in \mathbb{R}^d \tag{4}$$

The second derivative under mutl-variable function is referred as Hessian matrix, defined in Equation 5.

$$H = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix} \in \mathbb{R}^{d \times d} \tag{5}$$

Therefore, talyer expansion under a multi-variable system is defined as

$$f(\mathbf{x}_k + \Delta\mathbf{x}) \approx f(\mathbf{x}_k) + \nabla f(\mathbf{x}_k)^T \Delta\mathbf{x} + \frac{1}{2}\Delta\mathbf{x}^T H(\mathbf{x}_k)\Delta\mathbf{x} \tag{6}$$

To find the optimal step size, the results of the derivative and recursive step are defined in Equation 7.

$$\begin{aligned}
&\frac{\partial}{\partial(\Delta\mathbf{x}_k)}\left[f(\mathbf{x}_k)+\nabla f(\mathbf{x}_k)^T\Delta\mathbf{x}_k+\frac{1}{2}\Delta\mathbf{x}_k^T H(\mathbf{x}_k)\Delta\mathbf{x}_k\right]=\nabla f(\mathbf{x}_k)+H(\mathbf{x}_k)\Delta\mathbf{x}_k=\mathbf{0}\\
&\Delta\mathbf{x}_k=-H(\mathbf{x}_k)^{-1}\nabla f(\mathbf{x}_k),\quad \mathbf{x}_{k+1}=\mathbf{x}_k-H(\mathbf{x}_k)^{-1}\nabla f(\mathbf{x}_k)
\end{aligned}\tag{7}$$

The main problem of Newton's method is the necessity of calculating the Hessian matrix and its inverse. For n-dimensional data, the time complexity of a unit step is $O(n^3)$. In addition, if the inverse of the Hessian matrix is not defined, this method may not be applied.

1.2 BFGS optimization

BFGS (Broyden–Fletcher–Goldfarb–Shanno) optimization method uses the second derivative to find optimal points. The "BFGS" means names of relative people. The main problem of calculation of Hessian and its inverse. At first, the position displacement vector, gradient displacement vector, and approximate inverse Hessian matrix for each step are defined in Equation 8.

$$\begin{aligned}
&\mathbf{s}_k:=\mathbf{x}_{k+1}-\mathbf{x}_k\leftarrow \text{ position displacement vector}\\
&\mathbf{y}_k=\nabla f(\mathbf{x}_{k+1})-\nabla f(\mathbf{x}_k)\leftarrow \text{ gradient displacement vector}\\
&B_k^{-1}\approx H_k^{-1}\leftarrow \text{ approximate inverse Hessian matrix},\quad B_0^{-1}=I
\end{aligned}\tag{8}$$

As approximate Hessian, B_k^{-1} should satisfy secant condition, symmetry, and positive definiteness in Equation 9.

$$\begin{aligned}
&B_{k+1}\mathbf{s}_k=\mathbf{y}_k\quad\Longleftrightarrow\quad B_{k+1}^{-1}\mathbf{y}_k=\mathbf{s}_k\leftarrow \text{ Secant condition}\\
&B_{k+1}^{-1}=(B_{k+1}^{-1})^T\leftarrow \text{ Symmetry}\\
&\mathbf{v}^TA\mathbf{v}>0\quad(\forall\mathbf{v}\neq\mathbf{0})\leftarrow \text{ Positive definiteness}
\end{aligned}\tag{9}$$

The main concept of matrix update is a rank-2 update in Equation 10.

$$B_{k+1}=B_k+\alpha\mathbf{u}\mathbf{u}^T+\beta\mathbf{v}\mathbf{v}^T\quad \mathbf{u},\ \mathbf{v}\in\mathbb{R}^n,\ \alpha,\ \beta\in\mathbb{R}\tag{10}$$

The main concept is the secant condition that results in each coefficient in Equation 11.

$$\begin{aligned}
B_{k+1}\mathbf{s}_k&=B_k\mathbf{s}_k+\alpha\mathbf{u}\mathbf{u}^T\mathbf{s}_k+\beta\mathbf{v}\mathbf{v}^T\mathbf{s}_k=\mathbf{y}_k\\
&=B_k\mathbf{s}_k+\big(\alpha(\mathbf{u}^T\mathbf{s}_k)\big)\,\mathbf{u}+\big(\beta(\mathbf{v}^T\mathbf{s}_k)\big)\,\mathbf{v},\quad \mathbf{u}:=B_k\mathbf{s}_k,\ \mathbf{v}:=\mathbf{y}_k\\
&=B_k\mathbf{s}_k+\alpha(\mathbf{s}_k^TB_k\mathbf{s}_k)(B_k\mathbf{s}_k)+\beta(\mathbf{y}_k^T\mathbf{s}_k)\mathbf{y}_k\\
\mathbf{0}&=\left[1+\alpha(\mathbf{s}_k^TB_k\mathbf{s}_k)\right]B_k\mathbf{s}_k+\left[\beta(\mathbf{y}_k^T\mathbf{s}_k)-1\right]\mathbf{y}_k\\
\therefore\alpha&=-\frac{1}{\mathbf{s}_k^TB_k\mathbf{s}_k},\quad \beta=\frac{1}{\mathbf{y}_k^T\mathbf{s}_k}\\
B_{k+1}&=B_k-\frac{B_k\mathbf{s}_k\mathbf{s}_k^TB_k}{\mathbf{s}_k^TB_k\mathbf{s}_k}+\frac{\mathbf{y}_k\mathbf{y}_k^T}{\mathbf{y}_k^T\mathbf{s}_k}
\end{aligned}\tag{11}$$

To induce the inverse matrix, the main lemma (Rank-1 inverse form) is shown in Equation 12.

$$(A+\mathbf{u}\mathbf{v}^T)^{-1}=A^{-1}-\frac{A^{-1}\mathbf{u}\mathbf{v}^TA^{-1}}{1+\mathbf{v}^TA^{-1}\mathbf{u}}\tag{12}$$

Therefore, for the inverse matrix, the recursive form is defined in Equation 13.

$$B_{k+1}^{-1} = (I - \rho_k \mathbf{s}_k \mathbf{y}_k^T) B_k^{-1} (I - \rho_k \mathbf{y}_k \mathbf{s}_k^T) + \rho_k \mathbf{s}_k \mathbf{s}_k^T, \quad \rho_k = \frac{1}{\mathbf{y}_k^T \mathbf{s}_k}$$
$$\mathbf{p}_k = -B_k^{-1} \nabla f(\mathbf{x}_k), \quad \mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k \tag{13}$$

Although BFGS optimization decreases time complexity to $O(n^2)$, when ρ_k is negative or unstble, some numerical problem may occur.

2 L-BFGS

2.1 Background

For BFGS, to save $N \times N$ matrix may burden the usage of memory. Instead, just save some vector and express these vector as Hessian approximation, the usage of memory can be reduced, which contribute for efficiency and lightweight design.

$$\text{BFGS } (O(N^2) \text{ Space}) : \text{Full Inverse Hessian Matrix } B_k^{-1} \in \mathbb{R}^{100{,}000 \times 100{,}000}$$
$$\implies 10^{10} \text{ elements} \times 8 \text{ bytes (float64)} \approx \mathbf{80} \text{ GB}$$

$$\text{L-BFGS } (O(mN) \text{ Space}) : 2m \text{ Difference Vectors } \{\mathbf{s}_i, \mathbf{y}_i\}_{i=k-m}^{k-1} \in \mathbb{R}^{100{,}000} \quad (m = 10)$$
$$\implies 20 \times 100{,}000 \text{ elements} \times 8 \text{ bytes (float64)} \approx \mathbf{16} \text{ MB}$$

2.2 Method

The objective of L-BFGS is to find not Hessian matrix, but product of Hessian matrix and gradient in Equation 14.

$$B_{k+1}^{-1} \mathbf{g}_{k+1} = \left[(I - \rho_k \mathbf{s}_k \mathbf{y}_k^T) \, H_0^{k+1} \, (I - \rho_k \mathbf{y}_k \mathbf{s}_k^T) + \rho_k \mathbf{s}_k \mathbf{s}_k^T \right] \mathbf{g}_{k+1} \in \mathbb{R}^n, \; H_0^{k+1} = \frac{\mathbf{s}_k^T \mathbf{y}_k}{\mathbf{y}_k^T \mathbf{y}_k} I, \; \mathbf{g}_k = \nabla f(\mathbf{x}_k) \tag{14}$$

2.3 L-BFGS Derivation and Two-Loop Recursion

2.3.1 Inverse Hessian Update Formulation ($m = 1$)

The objective is to compute the search direction vector $\mathbf{r} = B_{k+1}^{-1} \mathbf{g}_{k+1}$ without explicitly constructing the inverse Hessian matrix B_{k+1}^{-1}. Substituting the base matrix $H_0^{k+1} = \gamma_{k+1} I$ into the BFGS inverse update formula yields:

$$B_{k+1}^{-1} \mathbf{g}_{k+1} = \left[(I - \rho_k \mathbf{s}_k \mathbf{y}_k^T) \, (\gamma_{k+1} I) \, (I - \rho_k \mathbf{y}_k \mathbf{s}_k^T) + \rho_k \mathbf{s}_k \mathbf{s}_k^T \right] \mathbf{g}_{k+1} \tag{15}$$

where $\rho_k = \frac{1}{\mathbf{y}_k^T \mathbf{s}_k}$.

2.3.2 Base Scaling Factor γ_{k+1}

To scale the identity matrix I according to the local curvature of the objective function, the most recent secant condition $\gamma_{k+1} \mathbf{y}_k \approx \mathbf{s}_k$ is used. Multiplying both sides by $\mathbf{y}_k^T$ yields:

$$\gamma_{k+1} (\mathbf{y}_k^T \mathbf{y}_k) = \mathbf{s}_k^T \mathbf{y}_k \implies \gamma_{k+1} = \frac{\mathbf{s}_k^T \mathbf{y}_k}{\mathbf{y}_k^T \mathbf{y}_k} \tag{16}$$

2.3.3 Step-by-Step Algebraic Decomposition

Step 1: Backward Pass (Loop 1 generation)
Multiplying the rightmost term $(I - \rho_k \mathbf{y}_k \mathbf{s}_k^T)$ by $\mathbf{g}_{k+1}$ and factoring out the scalar coefficient α_k:

$$\begin{aligned}(I - \rho_k \mathbf{y}_k \mathbf{s}_k^T)\mathbf{g}_{k+1} &= \mathbf{g}_{k+1} - \mathbf{y}_k(\rho_k \mathbf{s}_k^T \mathbf{g}_{k+1}) \\ \alpha_k &= \rho_k \mathbf{s}_k^T \mathbf{g}_{k+1} \\ \mathbf{q} &= \mathbf{g}_{k+1} - \alpha_k \mathbf{y}_k\end{aligned} \tag{17}$$

Step 2: Base Scaling
Applying the base matrix scaling to the transformed gradient vector $\mathbf{q}$:

$$\mathbf{r} = \gamma_{k+1} I \mathbf{q} = \gamma_{k+1} \mathbf{q} \tag{18}$$

Step 3: Forward Pass (Loop 2 generation)
Multiplying the remaining terms by $\mathbf{r}$ and $\mathbf{g}_{k+1}$, then factoring out the scalar coefficient β:

$$\begin{aligned}B_{k+1}^{-1}\mathbf{g}_{k+1} &= (I - \rho_k \mathbf{s}_k \mathbf{y}_k^T)\mathbf{r} + \rho_k \mathbf{s}_k \mathbf{s}_k^T \mathbf{g}_{k+1} \\ &= \mathbf{r} - \mathbf{s}_k(\rho_k \mathbf{y}_k^T \mathbf{r}) + \mathbf{s}_k(\rho_k \mathbf{s}_k^T \mathbf{g}_{k+1}) \\ \beta &= \rho_k \mathbf{y}_k^T \mathbf{r} \\ \mathbf{r} &\leftarrow \mathbf{r} + \mathbf{s}_k(\alpha_k - \beta)\end{aligned} \tag{19}$$

2.3.4 Full Iteration Step Definition

Given the m most recent vector pairs $\{\mathbf{s}_i, \mathbf{y}_i\}_{i=k-m}^{k-1}$, one full iteration step of L-BFGS is defined as follows:

$$\begin{aligned}&\text{[1. Two-Loop Recursion]} \\ &\quad \mathbf{q} \leftarrow \mathbf{g}_k \\ &\quad \text{For } i = k-1 \text{ down to } k-m: \\ &\qquad \alpha_i = \rho_i \mathbf{s}_i^T \mathbf{q} \\ &\qquad \mathbf{q} \leftarrow \mathbf{q} - \alpha_i \mathbf{y}_i \\ &\quad \gamma_k = \frac{\mathbf{s}_{k-1}^T \mathbf{y}_{k-1}}{\mathbf{y}_{k-1}^T \mathbf{y}_{k-1}} \\ &\quad \mathbf{r} \leftarrow \gamma_k \mathbf{q} \\ &\quad \text{For } i = k-m \text{ up to } k-1: \\ &\qquad \beta = \rho_i \mathbf{y}_i^T \mathbf{r} \\ &\qquad \mathbf{r} \leftarrow \mathbf{r} + \mathbf{s}_i(\alpha_i - \beta) \\ &\quad \mathbf{p}_k = -\mathbf{r} \\ &\text{[2. Position Update and Memory Refresh]} \\ &\quad \mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k \quad (\text{where } \alpha_k \text{ is determined via Line Search}) \\ &\quad \mathbf{s}_k = \mathbf{x}_{k+1} - \mathbf{x}_k \\ &\quad \mathbf{y}_k = \mathbf{g}_{k+1} - \mathbf{g}_k \\ &\quad \mathcal{M}_{k+1} = \{\mathcal{M}_k \setminus \{(\mathbf{s}_{k-m}, \mathbf{y}_{k-m})\}\} \cup \{(\mathbf{s}_k, \mathbf{y}_k)\}\end{aligned} \tag{20}$$

References

[1] A. Quarteroni, R. Sacco, F. Saleri, Numerical mathematics, Vol. 37, Springer Science & Business Media, 2006.